1 Introducción a la gestión de la ingeniería de software

Semana 1 · Hoja de ejercicios

Nombre _____ Fecha _____

Ejercicios

Muestra tu procedimiento. Cada ejercicio trae una pista al final: intenta primero sin ella.

1. Encuentra la brecha Calentamiento · Lección 1

Un cliente dice "haz el reporte más rápido". Escribe dos preguntas que harías antes de programar.

Pista: ¿Qué significa "más rápido" para ellos: segundos, minutos o otro conjunto de datos?

2. Escribe un requisito comprobable Práctica · Lección 1

Reescribe "el inicio de sesión debe ser seguro" como un requisito con una condición medible.

Pista: Nombra la condición, el actor y el resultado esperado.

3. Elige un modelo Calentamiento · Lección 2

Un formulario oficial tiene reglas legales fijas que no cambiarán en dos años. ¿Qué modelo encaja mejor y por qué?

Pista: Piensa cuánto cambio esperas después de firmar los requisitos.

4. Dibuja tu propio historial de líneas base Práctica · Lección 2

Un equipo construye una app de chat en tres iteraciones: inicio de sesión, mensajes y notificaciones. Nombra la línea base y el incremento tras la iteración 2.

Pista: Una línea base es una versión funcional; el incremento es la diferencia con la anterior.

5. Inventaría los elementos de configuración Calentamiento · Lección 3

Enumera cinco elementos de configuración de una app móvil que no sean código fuente.

Pista: Piensa en documentos, pruebas, configuración de compilación y versiones.

6. Rastrea un error hasta una línea base Desafío · Lección 3

Producción muestra un error que no estaba en la compilación de prueba. Describe las cuatro preguntas de la GCS que responderías para hallar la causa.

Pista: Identificación, control de versiones, control de cambios, contabilidad de estado.

7. Corrige el mensaje de commit Calentamiento · Lección 4

Reescribe estos mensajes en estilo imperativo: "cosas", "Fixed the bug.", "Added a new function for sums".

Pista: Empieza con un verbo, en minúscula, sin punto final, unos 50 caracteres.

8. Divide un commit Práctica · Lección 4

Un commit mezcla una corrección, un cambio de nombre y una prueba nueva. Describe cómo lo dividirías en tres commits lógicos.

Pista: Prepara los archivos relacionados juntos y haz commit de cada grupo por turno.

9. Predice el estado Calentamiento · Lección 5

Editas a.txt, ejecutas git add a.txt y luego editas a.txt otra vez. ¿La segunda edición está preparada? ¿Qué muestra git status?

Pista: La preparación captura el archivo en el momento de ejecutar git add.

10. Lee el grafo Práctica · Lección 5

Ejecuta git log --oneline --graph --all en la carpeta de prueba tras el Laboratorio 2. Explica en una frase por qué main y feature-sum apuntan al mismo commit.

Pista: ¿Qué pasa cuando la rama destino no tiene commits nuevos?

11. Resuelve según la intención Desafío · Lección 5

Dos ramas cambiaron la misma función: una renombró un parámetro y la otra añadió validación. Describe la resolución que quieres y los comandos.

Pista: Conserva ambas intenciones, luego prepara y haz commit.

12. Ordena el flujo de trabajo Calentamiento · Lección 6

Ordena estos pasos: fusionar, abrir solicitud de cambio, subir la rama, revisar y aprobar, crear la rama.

Pista: No puedes abrir una solicitud antes de que la rama exista en el remoto.

13. Recupérate de un push rechazado Desafío · Lección 6

Un compañero subió cambios a main mientras trabajabas. Tu git push fue rechazado. Escribe la secuencia segura de comandos.

Pista: Primero fetch, luego integra y luego push. Nunca fuerces.

Comprobación rápida

Encierra una respuesta.

14. ¿Qué elemento es un elemento de configuración en la gestión de la configuración del software?

A) Un documento de requisitos B) La cafetera de la oficina C) Un calendario personal

15. ¿Qué afirmación sobre Git es correcta?

A) Git es un sistema de control de versiones distribuido; cada clon contiene el historial completo.

B) Git no puede registrar un commit sin una conexión a internet a GitHub.

C) Git y GitHub son dos nombres para la misma herramienta.

16. En un proceso iterativo e incremental, ¿qué representa la diferencia entre dos líneas base sucesivas?

A) Un incremento B) Un registro de riesgos C) Solo la entrega final

17. Un compañero dice que la última versión se comporta incorrectamente. ¿Qué pregunta harías primero?

A) ¿Qué elementos de configuración y versiones están en esa línea base?

B) ¿Quién tiene la computadora más rápida? C) ¿Podemos borrar el repositorio y empezar de nuevo?

18. Quieres que un compañero revise tu cambio de inicio de sesión antes de que llegue a main. ¿Qué haces?

A) Subo una rama de funcionalidad y abro una solicitud de cambio para revisión

B) Subo directamente a main y pido comentarios después C) Envío por correo un zip del proyecto

19. Dos ramas cambiaron la misma línea de app.js. La fusión se detiene. ¿Qué pasó?

A) Un conflicto de fusión; una persona debe elegir el contenido final B) Git borró el repositorio

C) El remoto está desconectado

20. git push es rechazado porque el remoto tiene commits que no tienes localmente. ¿Cuál es la solución más segura?

A) Ejecutar git fetch, integrar los cambios remotos (merge o rebase) y luego volver a hacer push

B) Ejecutar git push --force para sobrescribir el remoto C) Borrar tu repositorio local y clonar de nuevo

21. git commit falla con "Author identity unknown". ¿Qué debes hacer?

A) Configurar tu nombre y correo con git config --global user.name y user.email B) Borrar la carpeta .git

C) Hacer commit con git add -f

Respuestas

1. Ejemplo: "¿Qué reporte, y cuánto tarda hoy frente a la meta?" y "¿Quién lo usa y con qué frecuencia?". Cada respuesta convierte un deseo vago en un requisito comprobable.
2. Ejemplo: "Tras 5 intentos fallidos en 10 minutos, la cuenta se bloquea 15 minutos y el usuario recibe un correo." Ahora un tester puede comprobarlo.
3. Puede encajar un proceso secuencial (tipo cascada), porque los requisitos son estables y los errores tempranos serían caros. Un proceso iterativo también sirve si el equipo quiere retroalimentación temprana de la interfaz.
4. La línea base 2 es inicio de sesión más mensajes, funcional y documentada. El incremento es la función de mensajes comparada con la línea base 1.
5. Ejemplos: especificación de requisitos, archivos de diseño de la interfaz, plan y scripts de prueba, archivos de configuración de compilación, notas de versión, manual de usuario.
6. ¿Qué versiones exactas están en producción (identificación)? ¿Qué cambió entre la compilación de prueba y producción (control de versiones)? ¿Qué cambios se aprobaron y quién (control de cambios)? ¿La compilación de producción es la registrada en la línea base (contabilidad de estado)?
7. fix login redirect loop | fix bug … (nombra el error) | add sum function. Los verbos específicos ayudan a leer el historial rápido.
8. Prepara los archivos de la corrección y haz commit; prepara los archivos renombrados y haz commit; prepara la prueba y haz commit. Ahora cada commit se puede revisar o revertir por separado.
9. No: la segunda edición no está preparada. git status muestra a.txt en "Changes to be committed" (la primera versión) y en "Changes not staged for commit" (la edición nueva).
10. main no tenía commits nuevos desde que se separó feature-sum, así que Git movió el puntero hacia adelante (fast-forward) en lugar de crear un commit de fusión.
11. Edita la función para usar el nuevo nombre del parámetro e incluir la validación. Quita todos los marcadores, ejecuta git add sobre el archivo y luego git commit para terminar la fusión.
12. Crear la rama → subir la rama → abrir la solicitud de cambio → revisar y aprobar → fusionar.
13. git fetch origin, luego git pull origin main (o git rebase origin/main), resuelve conflictos, ejecuta pruebas y luego git push origin main.
14. **A** — Cualquier artefacto que forme parte del producto o de su documentación puede ser un elemento de configuración. Los requisitos son un ejemplo central.
15. **A** — Cada clon tiene el repositorio y el historial completos, así que puedes hacer commits sin conexión.
16. **A** — Por definición, la diferencia entre dos líneas base es un incremento.
17. **A** — Antes de depurar, identifica exactamente qué versiones se publicaron. La GCS hace que esto sea trazable.
18. **A** — Una rama de funcionalidad con solicitud de cambio permite que quien revisa vea el diff y apruebe antes de fusionar.
19. **A** — Git no puede elegir entre dos ediciones distintas de las mismas líneas, así que marca el conflicto para que lo resuelvas.
20. **A** — Integra primero el trabajo remoto. Un push forzado puede borrar los commits de un compañero y debe evitarse en ramas compartidas.
21. **A** — Git guarda el nombre y el correo del autor en cada commit, así que ambos deben configurarse primero.